

1. Which Python keyword indicates the start of a function definition?

1 / 1 point

Select the correct option.

- ☒ def
- ☐ continue
- ☐ sweet
- ☐ help

 Nice work

Correct! The **def** keyword is used to define a function in Python.

2. In Python, how do you indicate the end of the block of code that makes up the function?

1 / 1 point

Select the correct option.

- ☒ You de-indent a line of code to the same indent level as the **def** keyword.
- ☐ You put the colon character (:) in the first column of a line.
- ☐ You add the matching curly brace that was used to start the function }.
- ☐ You add a line that has at least 10 dashes.

 Nice work

Correct! In Python, de-indentation to the same level as the **def** keyword indicates the end of the function block.

3. In Python what is the **input()** feature best described as?

1 / 1 point

Select the correct option.

- ☐ A reserved word
- ☐ A user-defined function
- ☐ The central processing unit
- ☒ A built-in function

 Nice work

Correct! The **input()** function is a built-in function in Python used to read input from the user.

4. What does the following code print out?

1 / 1 point

```
1 def thing():
2     print('Hello')
3
4     print('There')
```

Select the correct option.

- ☐ def
- ☐ thing
- ☒ thing
- ☐ Hello
- ☐ There
- ☐ thing
- ☒ There

 Nice work

Correct! Since the **thing()** function is not called, neither **print('Hello')** nor **print('There')** runs.

5. In the following Python code, which of the following is an "argument" to a function?

1 / 1 point

```
1 x = 'banana'
2 y = max(x)
3 print(y)
```

Select the correct option.

- ☐ 'banana'
- ☒ x and y
- ☐ print
- ☐ max

 Nice work

Not quite. **x** is passed as an argument to **max()**, but **y** is not an argument but the result.

6. What will the following Python code print out?

1 / 1 point

```
1 def func(x) :
2     print(x)
3
4     func(10)
5     func(20)
```

Select the correct option.

- ☐ x
- ☐ 10
- ☐ x
- ☐ 20
- ☒ 10
- ☐ 20
- ☐ def
- ☐ x
- ☐ func
- ☐ func
- ☐ x
- ☐ 20

 Nice work

Correct! The function **func()** is called twice with the values 10 and 20, respectively, printing each value.

7. Which line of the following Python program will never execute?

1 / 1 point

```
1 def stuff():
2     print('Hello')
3     return
4     print('World')
5
6     stuff()
```

Select the correct option.

- ☒ print('World')
- ☐ print('Hello')
- ☐ return
- ☐ def stuff():

 Nice work

Correct! The **print('World')** will never execute because it is after the return statement.

8. What will the following Python program print out?

1 / 1 point

```
1 def greet(lang):
2     if lang == 'es':
3         return 'Hola'
4     elif lang == 'fr':
5         return 'Bonjour'
6     else:
7         return 'Hello'
8
9 print(greet('fr'), 'Michael')
```

Select the correct option.

- ☒ Bonjour Michael
- ☐ def
- ☐ Hola
- ☐ Bonjour
- ☐ Hello
- ☐ Michael
- ☐ Hola Michael
- ☐ Hola
- ☐ Bonjour
- ☐ Hello
- ☐ Michael

 Nice work

Correct! The **greet** function returns 'Bonjour' for '**fr**', and 'Michael' is printed next to it.

9. What does the following Python code print out? (Note that this is a bit of a trick question and the code has what many would consider to be a flaw/bug - so read carefully).

1 / 1 point

```
1 def addtwo(a, b):
2     added = a + b
3     return a
4
5 x = addtwo(2, 7)
6 print(x)
```

Select the correct option.

- ☒ 2
- ☐ 7
- ☐ 14
- ☐ 9

 Nice work

Correct! The function should return **added**, but it mistakenly returns **a**.

10. What is the most important benefit of writing your own functions?

1 / 1 point

Select the correct option.

- ☐ Following the rule that no function can have more than 10 statements in it
- ☐ To avoid having more than 10 lines of sequential code without an indent or de-indent
- ☐ Following the rule that whenever a program is more than 10 lines you must use a function
- ☒ Avoiding writing the same non-trivial code more than once in your program

 Nice work

Correct! Deciding to write a function for your own use is something that you see when your programs reach a certain level of complexity. There are no strict rules about the length or number of statements, but functions help manage complexity by encapsulating repetitive code.